

Introducción C#

Módulo 1

Operadores

¿Qué es un operador?

Un **operador** en C# es un símbolo formado por uno o más caracteres que permite realizar una determinada operación entre uno o más datos y produce un resultado.

A continuación, se describen cuáles son los operadores incluidos en el lenguaje, clasificados según el tipo de operaciones que permiten realizar. Hay que tener en cuenta que C# permite la redefinición del significado de la mayoría de los operadores según el tipo de dato sobre el que se apliquen, así que lo que aquí se cuenta se corresponde con los usos más comunes de los mismos.



Un operador es una expresión que produce otro valor (así como las funciones o construcciones que devuelven un valor).

Existen operadores de comparación, de negación o de incremento y decremento.

Las operaciones matemáticas se comportan de igual manera en PHP. Las operaciones `*` y `/` tienen precedencia sobre la suma y la resta y se pueden utilizar paréntesis para expresiones más complejas.

Un operador es un símbolo especial que indica al compilador que debe efectuar una operación matemática o lógica.

En todo lenguaje de programación existe un conjunto de operadores que permiten realizar operaciones con los datos. Nos referimos a operaciones aritméticas, comparaciones, operaciones lógicas y otras operaciones (por ejemplo, concatenación de cadenas, operaciones algebraicas entre valores números, etc.).

Operaciones aritméticas

Los operadores aritméticos incluidos en C# son los típicos de suma (+), resta (-), producto (*), división (/) y módulo (%) También se incluyen operadores de “menos unario” (-) y “más unario” (+).

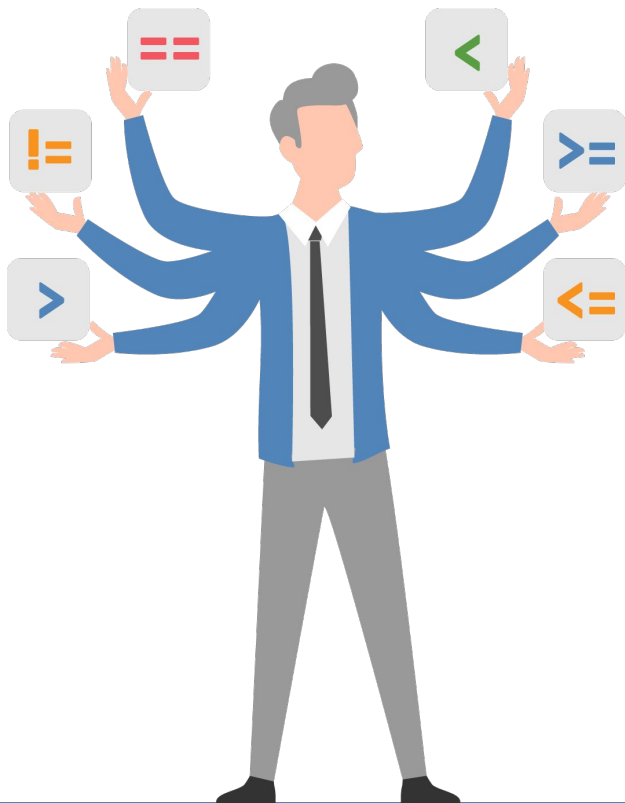
Operaciones lógicas

Se incluyen operadores que permiten realizar las operaciones lógicas típicas: “and” (&& y &), “or” (| | y |), “not” (!) y “xor” (^)

Los operadores && y | | se diferencia de & y | en que los primeros realizan evaluación perezosa y los segundos no. La evaluación perezosa consiste en que si el resultado de evaluar el primer operando permite deducir el resultado de la operación, entonces no se



evalúa el segundo y se devuelve dicho resultado directamente, mientras que la evaluación no perezosa consiste en evaluar siempre ambos operandos. Es decir, si el primer operando de una operación && es falso se devuelve **false** directamente, sin evaluar el segundo; y si el primer operando de una | | es verdadero se devuelve **true** directamente, sin evaluar el otro.



Operaciones relacionales

Se han incluido los tradicionales operadores de igualdad (==), desigualdad (!=), “mayor que” (>), “menor que” (<), “mayor o igual que” (>=) y “menor o igual que” (<=).

Operaciones de asignación

Para realizar asignaciones, se usa en C# el operador `=`, que además de realizar la asignación que se le solicita, devuelve el valor asignado. Por ejemplo, la expresión $a = b$ asigna a la variable a el valor de la variable b y devuelve dicho valor, mientras que la expresión $c = a = b$ asigna a las variables c y a el valor de b (el operador `=` es asociativo por la derecha).

También se han incluido operadores de asignación compuestos que permiten ahorrar tecleo a la hora de realizar asignaciones tan comunes como:

```
temperatura = temperatura + 15; // Sin usar asignación  
compuesta
```

```
temperatura += 15; // Usando asignación compuesta
```

Las dos líneas anteriores son equivalentes, pues el operador compuesto `+=` asigna a su primer operando el valor que tenía más el de su segundo operando (o sea, le suma el segundo operando) Como se ve, permite compactar bastante el código.

Aparte del operador de asignación compuesto `+=`, también se ofrecen operadores de asignación compuestos para la mayoría de los operadores binarios ya vistos. Estos son: `+=`, `-=`, `*=`, `/=`, `%=`, `&=`, `|=`, `^=`, `<<=` y `>>=`. Nótese que no hay versiones compuestas para los operadores binarios `&&` y `||`.

Otros operadores de asignación incluidos son los de incremento (++) y decremento (--).

Estos operadores permiten, respectivamente, aumentar y disminuir en una unidad el valor de la variable sobre el que se aplican. Así, estas líneas de código son equivalentes.

Si el operador ++ se coloca tras el nombre de la variable (como en el ejemplo) devuelve el valor de la variable antes de incrementarla, mientras que si se coloca antes, devuelve el valor de ésta tras incrementarla; y lo mismo ocurre con el operador --. Por ejemplo:

```
static void Main(string[] args)
{
    double temperatura = 0;
    temperatura = temperatura + 1; // Sin asignación compuesta ni incremento
    temperatura += 1; // Usando asignación compuesta
    temperatura++; // Usando incremento
}
```

```
c = b++; // Se asigna a c el valor de b y luego se incrementa b
```

```
c = ++b; // Se incrementa el valor de b y luego se asigna a c
```

La ventaja de usar los operadores ++ y -- es que en muchas máquinas son más eficientes que otras formas de realizar sumas o restas de una unidad, pues el compilador puede traducirlos en una única instrucción en código máquina.

Operaciones con cadenas

Para realizar operaciones de concatenación de cadenas se puede usar el mismo operador que para realizar sumas, ya que en C# se ha redefinido su significado para que cuando se aplique entre operandos que sean cadenas o que sean una cadena y un carácter lo que haga sea concatenarlos.

Por ejemplo, *"Hola"+" mundo"* devuelve *"Hola mundo"*, y *"Hola mund" + 'o'* también.



Operador condicional

Es el único operador incluido en C# que toma 3 operandos, y se usa así:

`<condición> ? <expresión1> : <expresión2>`

El significado del operando es el siguiente: se evalúa `<condición>`. Si es cierta, se devuelve el resultado de evaluar `<expresión1>` y, si es falsa, se devuelve el resultado de evaluar `<condición2>`. Un ejemplo de su uso es:

```
b = (a>0)? a : 0; // Suponemos a y b de tipos enteros
```

En este ejemplo, si el valor de la variable `a` es superior a 0 se asignará a `b` el valor de `a`, mientras que en caso contrario el valor que se le asignará será 0.

Hay que tener en cuenta que este operador es asociativo por la derecha, por lo que una expresión como `a?b:c?d:e` es equivalente a `a?b:(c?d:e)`

No hay que confundir este operador con la instrucción condicional **if** que se tratará en el *Tema Instrucciones*, pues aunque su utilidad es similar al de ésta, `?` devuelve un valor e **if** no.

¡Muchas gracias!

¡Sigamos trabajando!